

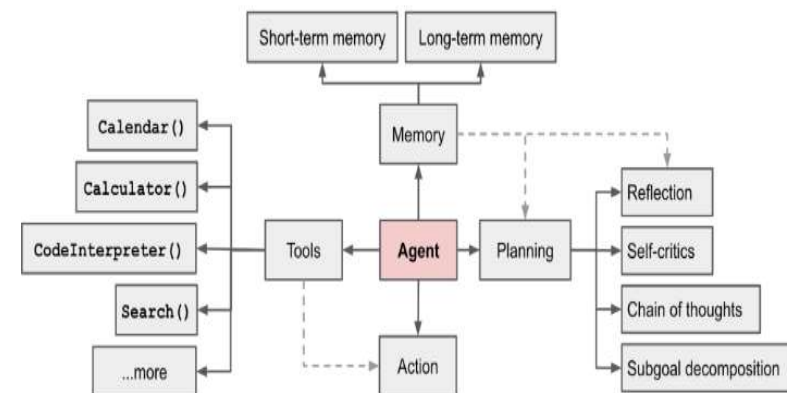
CN7050 – Intelligent Systems

Week 8: LLM Powered Autonomous Agents

Dr Shaheen Khatoon

Office:EB.1.99

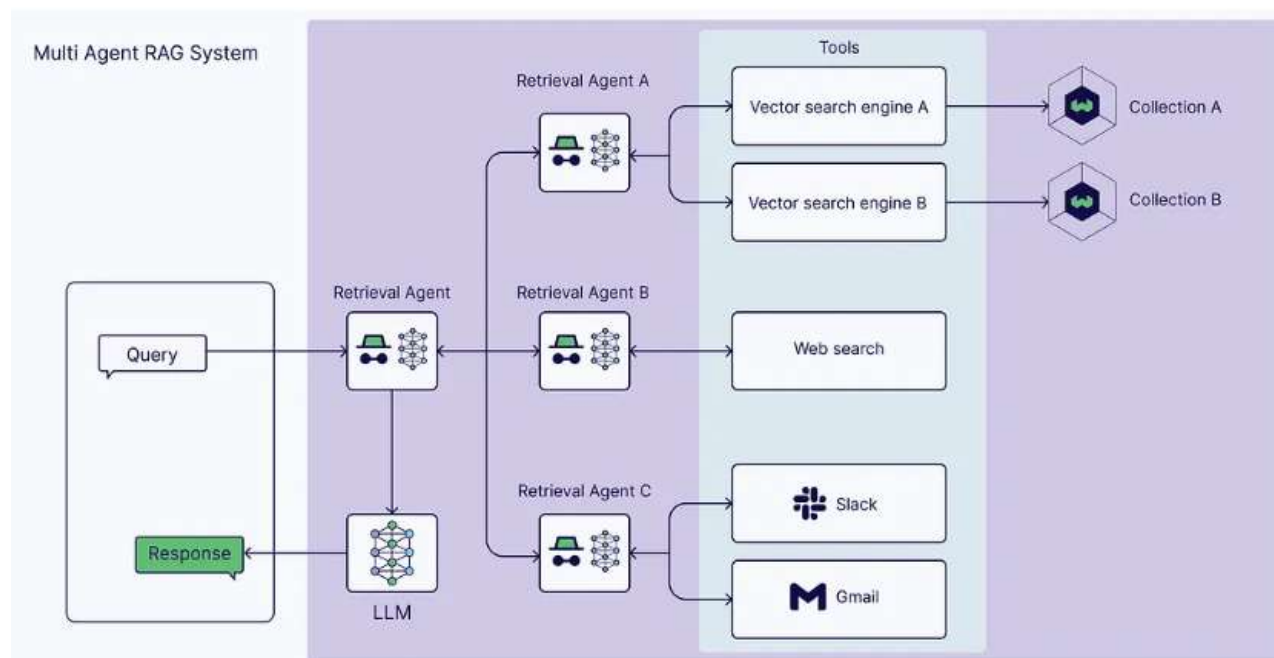
Email: s.khatoon@uel.ac.uk



Outline

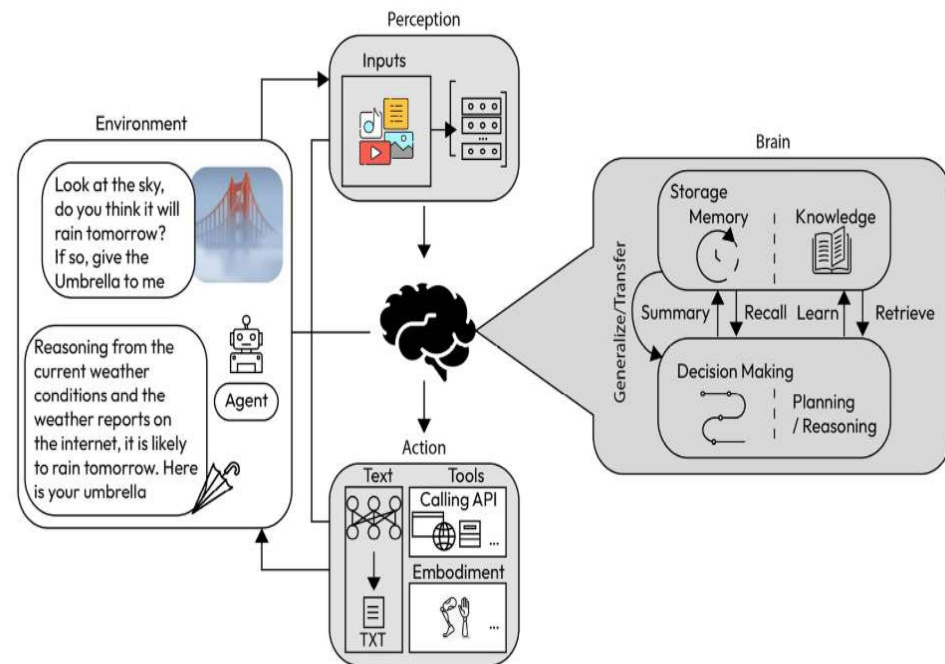
- Create agent which uses an **LLM** as its reasoning engine to interact with an environment through a set of available tools.
- Focuses on
 - **Extending the capabilities of LLMs** by enabling them to:
 - access, retrieve, and reason over external sources of knowledge
 - Begins with the creation of AI agents that can:
 - Interact with the web,
 - Retrieve live information,
 - and execute tasks beyond simple question answering.

Architecture of LLM Powered Agents



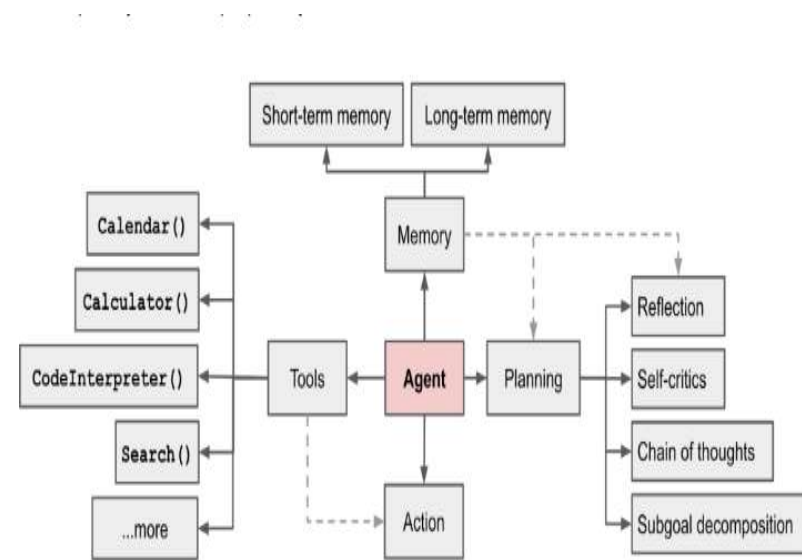
Conceptual framework of an LLM-based agents

- An **AI agent** is defined by its ability to perform an action, but it does not possess **desires and intentions**
 - **LLM (Brain):** The main component of the system that integrates information, stores it, and makes decisions
 - **Input (Perception):** The component that extends the model's capabilities in the perceptual domain, allowing the system to obtain information from different modalities (textual, auditory, and visual modalities)
 - **Tools (Action):** The component that enables the model to act and use tools to modify its surroundings (or respond to changes in the environment)



Core components of an LLM-Powered Agent

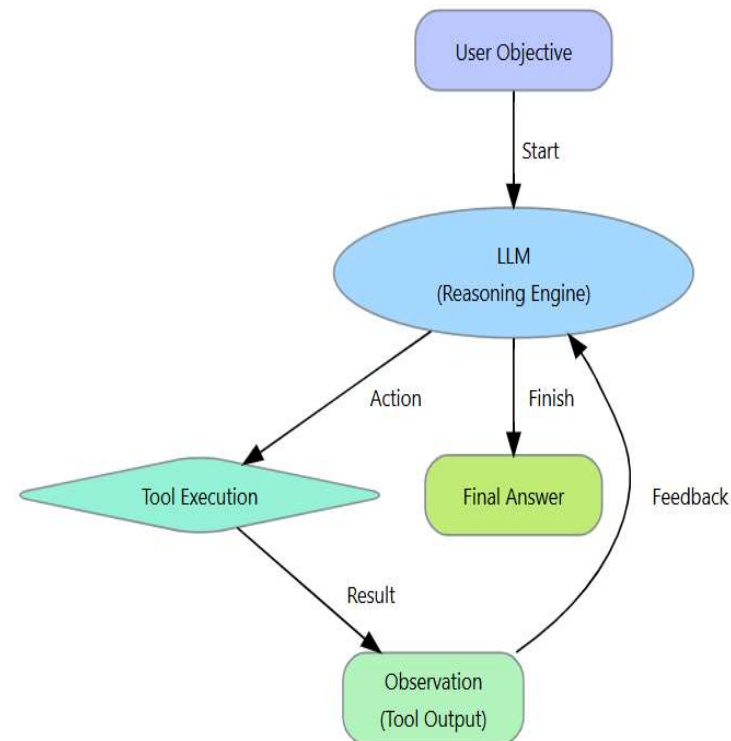
- **LLM with a Role and Task:** The foundational element, where the LLM is assigned a specific **role and task**, **guiding its interactions and decision-making processes**.
- **Memory:** Comprising **both short-term and long-term memory**, this component allows the agent to retain and utilize **past interactions and information**, facilitating **context-aware responses and learning over time**.
- **Planning:** Involves capabilities such as **reflection**, **self-critique**, and **query routing**, enabling the agent to **devise strategies and determine the necessary steps** to accomplish its assigned tasks effectively.
- **Tools:** External resources like **calculators**, **web search engines**, or **APIs** that the agent can utilize to enhance its functionality and provide accurate, **contextually relevant outputs**.



Core components of an AI agent

ReAct Framework

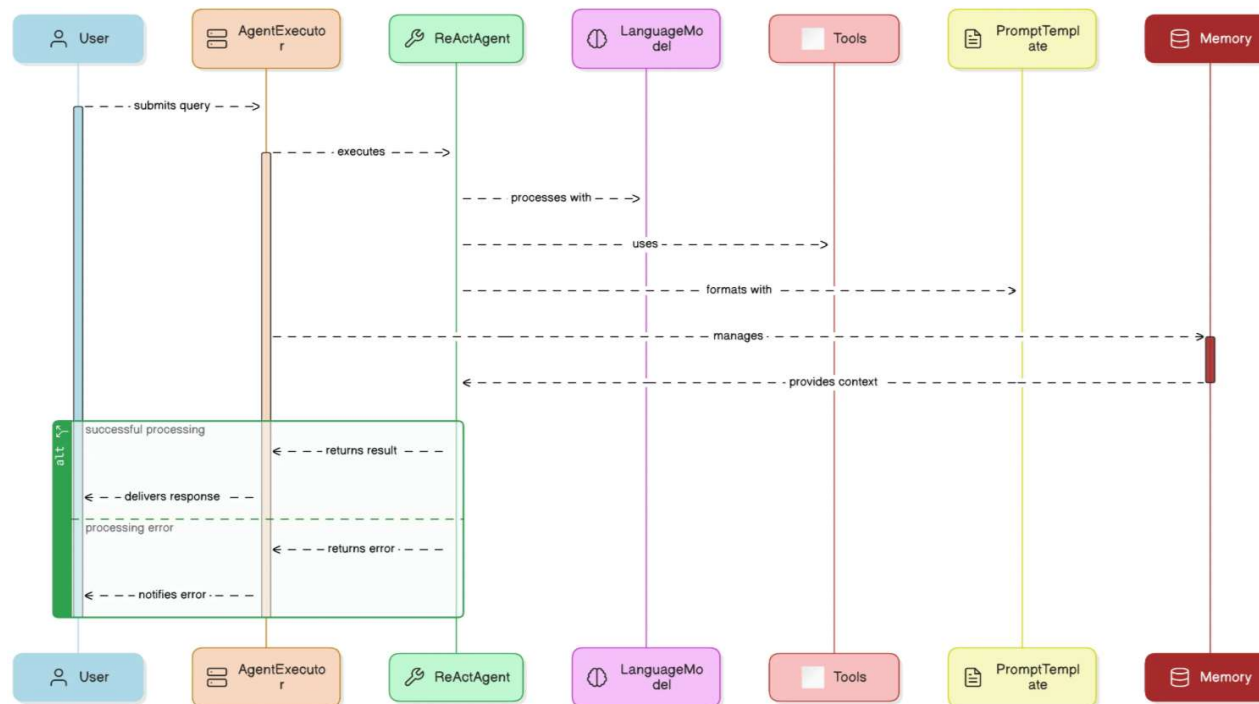
- A notable framework in this context is the **ReAct** framework, which stands for **Reason + Act**.
- This approach enables AI agents to handle sequential, multi-part queries while maintaining state through a cyclical process:
 1. **Thought:** Upon receiving a user query, the agent reasons about the next action to take.
 2. **Action:** The agent decides on an action and executes it, such as utilizing a tool or retrieving information.
 3. **Observation:** The agent observes the feedback resulting from its action.



The Agentic Loop: Think, Act, Observe

- At the heart of every agent is a **reasoning loop**.
 - This loop enables the agent to plan and execute **multi-step tasks** that may require accessing external information or performing calculations. The process generally follows these steps
- 1. **Objective:** The agent receives a high-level objective from the user
 - (e.g., "Who is the current CEO of Microsoft, and what was the stock price when they took over?").
- 2. **Thought:** The LLM, acting as the agent's brain, **analyzes the objective**. It considers the tools it has at its disposal and **formulates a plan**. It **might reason**,
 - *"To answer this, I first need to find out who the CEO is and when they started. A web search tool would be suitable for this."*
- 3. **Action:** The LLM outputs a specific action to take, indicating which tool to use and what input to provide.
 - For instance, it might decide to use a **Search tool** with the input "**current CEO of Microsoft**".
- 4. **Observation:** The **AgentExecutor** invokes the specified tool with the given input. The tool runs (e.g., performs a web search) and returns a result. **This result is called an observation.**
- 5. **Repeat:** The observation is passed back to the LLM. The LLM now has new information and re-evaluates its plan. It might think, "I have found that Satya Nadella became CEO on February 4, 2014. Now I need to find the stock price on that date." It then selects another action, perhaps using a financial data tool.

Query processing Sequence



Tools: The Agent's Capabilities

- If the agent is the brain, tools are its hands. A tool is a **function or service** that an agent can call to interact outside the LLM.
- An agent is only as capable as the tools it can use
- In **LangChain**, a tool is essentially a **function with a specific purpose**, packaged **with a name** and a **description** that the LLM can understand.
- Examples of tools include:
 - **Web Search**: A tool to query a search engine like Google or DuckDuckGo.
 - **Python REPL**: A tool that can execute arbitrary Python code.
 - **Database Query**: A tool for running SQL queries against a database.
 - **API Caller**: A generic tool for making requests to external APIs.

- See how to use tools:

<https://docs.langchain.com/oss/python/integrations/tools>

<https://docs.langchain.com/oss/python/langchain/tools>

Creating Search Tool

LangChain: Offer several tools for searching the internet.

For example:

- [DuckDuckGo](#)
 - [Google Serper](#)
 - [Wikipedia search etc.](#)
-
- DuckDuckGo tool allows to use the [DuckDuckGo search engine](#).
 - This search engine is free and does not track user data

Using DuckDuckGo

```
!pip install duckduckgo-search
from langchain.tools import DuckDuckGoSearchRun
from langchain.agents import Tool
tools = [
    Tool(
        name="DuckDuckGoSearch",
        func=ddg_search.run,
        description="A web search tool to extract
        information from Internet.",
    )
]
```

The most important part of a tool is its **description**. The LLM does not know how the tool's code works; it relies entirely on the tool's description to decide when and how to use it. A well-written description is specific and clearly explains what the tool does, what its expected input is, and what it returns.

Creating Search Tool

LangChain: Offer several tools for searching the internet.

For example:

- DuckDuckGo
- **Google Serper**
- Wikipedia search etc.
- Google Serper, a low-cost API enables you to use the Google search engine
- (you have to register at <https://serper.dev/> to get an API key)
- there is a **fee** for the service

Using Google Serper

```
#!/pip install google-serp-api
import os
SERPER_API_KEY = 'your_key'
os.environ["SERPER_API_KEY"] = SERPER_API_KEY
from langchain.utilities import GoogleSerperAPIWrapper
google_search = GoogleSerperAPIWrapper()
tools.append(
    Tool(
        name="Google Web Search",
        func=google_search.run,
        description="Google search tool to extract information from Internet.",
    )
)
```

Tool Selection by LLM agent's system

LangChain: Offer several tools for searching the internet.

For example:

- DuckDuckGo
 - Google Serper
 - **Wikipedia search** etc.
-
- can add Wikipedia as a reliable source of information:

Using Wikipedia

```
from langchain.tools import WikipediaQueryRun
from langchain.utilities import WikipediaAPIWrapper

wikipedia =
WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())

tools.append(
    Tool(
        name="Wikipedia Web Search",
        func=wikipedia.run,
        description="Useful tool to search Wikipedia.",
    )
)
```

The Agent: The Reasoning Engine

- "Agent" in LangChain is an abstraction that combines a **language model** with a carefully constructed **prompt**.
- This **prompt** is what transforms a general-purpose **LLM** into a **reasoning engine** capable of driving a task forward.
- The prompt instructs the LLM to follow a specific thought process "reasoning loop." A common pattern, used by frameworks like **ReAct (Reasoning and Acting)**, asks the model to break down its **thinking into distinct steps**:
 - **Thought:** The agent's internal monologue, where it analyzes the current situation, assesses progress towards the goal, and decides what to do next.
 - **Action:** The specific tool the agent decides to use to make progress.
 - **Action Input:** The parameters or query to pass to the chosen tool.
 - **Observation:** The result returned from executing the tool, which the agent will use in its next "Thought" step.
- The LLM generates text that follows this structure. The agent's sole responsibility is to produce the next thought and action based on the history of previous actions and observations.

The Reasoning Engine: Create prompt

- The prompt instructs the LLM to follow a specific thought process "reasoning loop."

```
1 from langchain.prompts import PromptTemplate
2
3 template = """Answer the following questions as best you can. You have access
4 to the following tools:
5 {tools}
6
7 Use the following format:
8
9 Question: the input question you must answer
10 Thought: you should always think about what to do
11 Action: the action to take, should be one of [{tool_names}]
12 Action Input: the input to the action
13 Observation: the result of the action
14 ... (this Thought/Action/Action Input/Observation can repeat N times)
15 Thought: I now know the final answer
16 Final Answer: the final answer to the original input question
17
18 Question: {input}
19 {agent_scratchpad}"""
20 prompt = PromptTemplate.from_template(template)
```

Initialize an agent and conduct query

Once we have the various tools, we can then initialize an agent and conduct a query:

The model will choose which results will be interesting to our query.

```
from langchain.agents import initialize_agent
agent = initialize_agent(
    tools, llm, agent="zero-shot-react-description",
    verbose=True
)

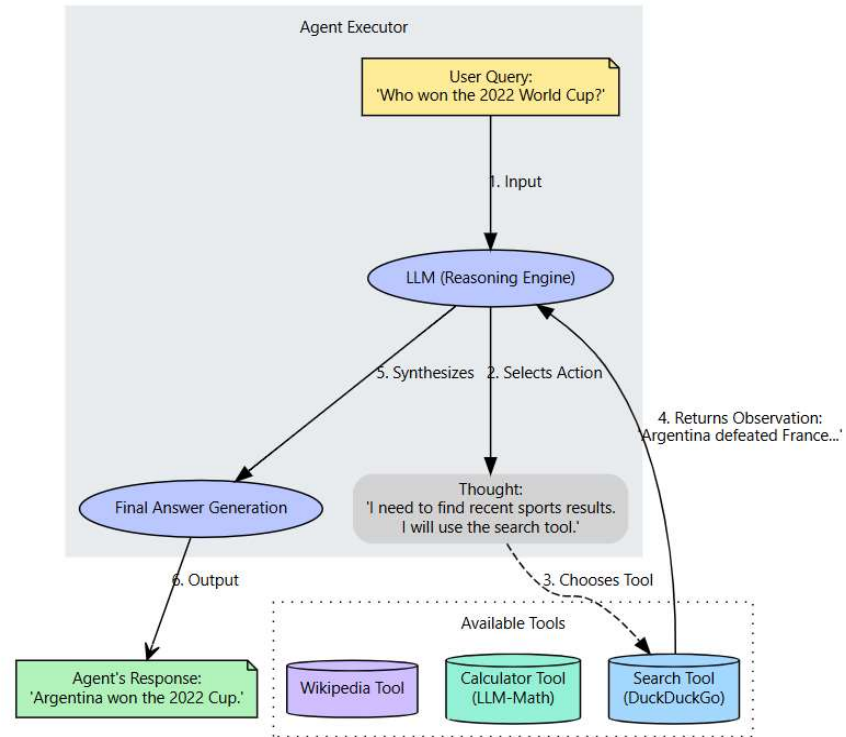
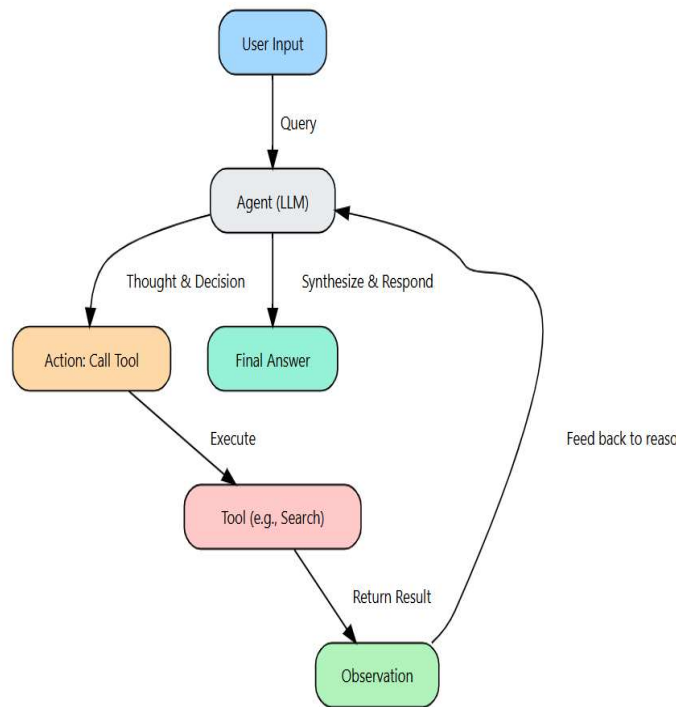
llm = "microsoft/Phi-3-mini-4k-instruct"

agent.run('Query')
```

```
# Define the query
query = "Who is the current president of Italy?
Who was the previous one?"

# Execute the agent query
response = agent.run(query)
print(response)
```

Execution loop



Typical Output

Agent execution:

thought, action, and observation

zero-shot-react-description

Zero-Shot means the agent is not given examples — it must reason from the tool descriptions and instructions alone.

ReAct *Reason + Act*, a framework where the model alternates between reasoning steps (“**Thought:**”) and actions (“**Action:** DuckDuckGo search”).

Description indicates it uses the tool descriptions to decide which tool to use.

verbose=True

it prints the agent's internal monologue, showing you exactly what it's thinking and doing at each step.

Use the following format:

Question: the input question you must answer

Thought: you should always think about what to do

Action: the action to take, should be one of [DuckDuckGo Search, Wikipedia Web Search]

Action Input: the input to the action

Observation: the result of the action

... (this Thought/Action/Action Input/Observation can repeat N times)

Thought: I now know the final answer

Final Answer: the final answer to the original input question

Begin!

Question: Who is the current president of Italy? Who was the previous one?

Thought: I need to find the current and previous presidents of Italy.

Action: DuckDuckGo Search

Action Input: "current president of Italy"

Observation: The search results indicate that the current president of Italy is Sergio Mattarella.

Thought: Now I need to find the previous president.

Action: DuckDuckGo Search

Action Input: "previous president of Italy"

Observation: The search results indicate that the previous president of Italy was Sergio Mattarella.

Thought: I now know the final answer.

Final Answer: The current president of Italy is Sergio Mattarella, and the previous one was also Sergio Mattarella.

Create_React_Object

- Another way to use ReAct framework by using Create_React_Object

```
1 from langchain.agents import AgentExecutor, create_react_agent
```

```
1 agent = create_react_agent(  
2     llm=llm,  
3     tools=tools,  
4     prompt=prompt  
5 )
```

```
1 agent_executor = AgentExecutor(  
2     agent=agent,  
3     tools=tools,  
4     verbose=True,  
5     max_iterations=5,  
6     handle_parsing_errors=True  
7 )
```

```
# Define the query
```

```
query = "Who is the current president of Italy? Who was the  
previous one?"
```

```
# Execute the agent query
```

```
response = agent_executor.invoke({"input": query})  
print(response["output"])
```

Use the following format:

Question: the input question you must answer

Thought: you should always think about what to do

Action: the action to take, should be one of [DuckDuckGo Search, Wikipedia Web Search]

Action Input: the input to the action

Observation: the result of the action

... (this Thought/Action/Action Input/Observation can repeat N times)

Thought: I now know the final answer

Final Answer: the final answer to the original input question

Begin!

Question: Who is the current president of Italy? Who was the previous one?

Thought: I need to find the current and previous presidents of Italy.

Action: DuckDuckGo Search

Action Input: "current president of Italy"

Observation: The search results indicate that the current president of Italy is Sergio Mattarella.

Thought: Now I need to find the previous president.

Action: DuckDuckGo Search

Action Input: "previous president of Italy"

Observation: The search results indicate that the previous president of Italy was Sergio Mattarella.

Thought: I now know the final answer.

Final Answer: The current president of Italy is Sergio Mattarella, and the previous one was also Sergio Mattarella.

Exploring Agent Type: ReAct

- **ReAct** framework, "**Reason and Act**," is one of the most common and effective general-purpose strategies for agents.
- The main idea is that the LLM explicitly verbalizes its reasoning process (**Thought**) before deciding on an Action. This chain-of-thought process makes the agent's behavior more interpretable and often leads to more reliable tool use.
- A ReAct agent's prompt template guides the LLM to generate text in a specific format:
 - Question: The user's query goes here.
 - Thought: The model's reasoning about what it needs to do next.
 - Action: The tool to use and the input for that tool.
 - Observation: The result from executing the tool.
 - ... (this Thought/Action/Observation cycle repeats)
 - Thought: I now have enough information to answer the user's question.
 - Final Answer: The final response to the user.

Agent Type: Self-Ask with Search

- Designed for question-answering tasks where a complex question needs to be broken down into simpler, factual sub-questions.
- It iteratively "asks itself" these simpler questions and uses a Search tool to find the answers. Once all the intermediate answers are gathered, it synthesizes them into a final response.
- The prompt for this agent encourages a different reasoning pattern:
 - Question: Who was the maternal grandfather of the 14th Dalai Lama?
 - Are follow up questions needed here: Yes.
 - **Follow up:** Who was the 14th Dalai Lama's mother?
 - **Intermediate answer:** The 14th Dalai Lama's mother was Diki Tsering.
 - **Follow up:** Who was Diki Tsering's father?
 - Intermediate answer: Diki Tsering's father was Choekyong Tsering.
 - So the final answer is: Choekyong Tsering.

Agent Type: Conversational Agents

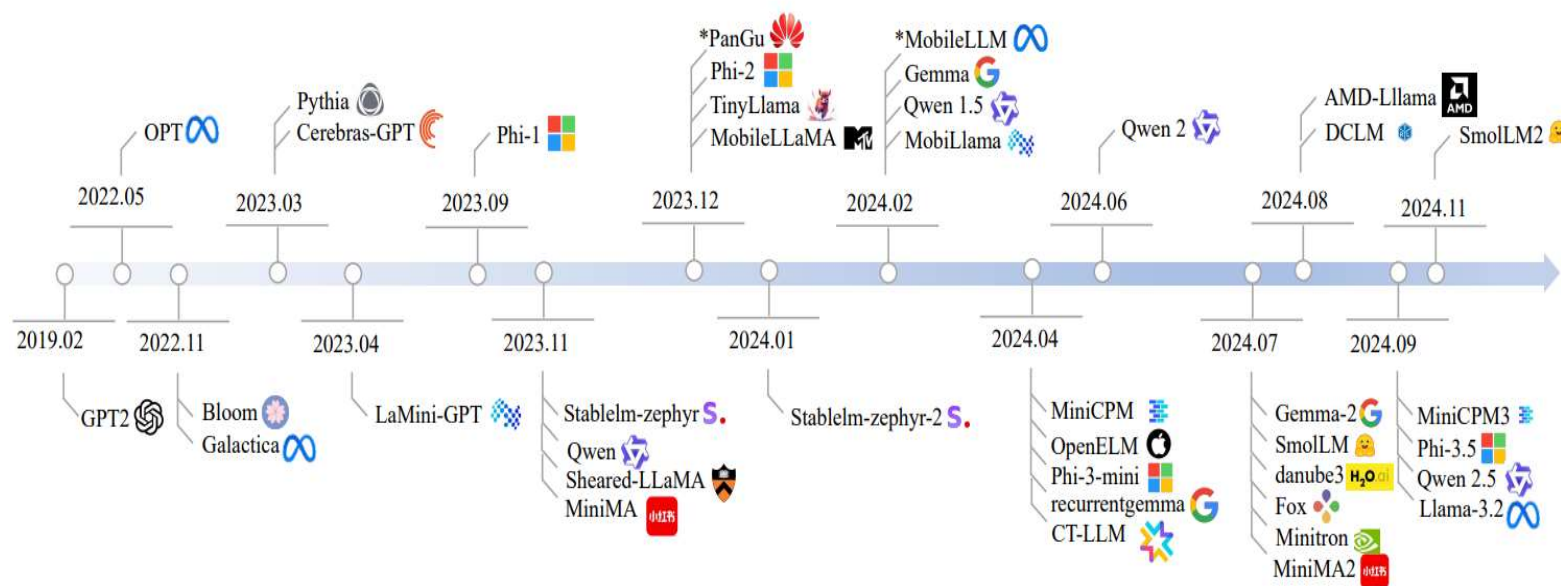
- Standard agents are typically stateless; they treat each query independently.
- **Conversational agents** are designed to overcome this limitation by **incorporating memory**.
- This allows them to remember previous interactions in a conversation and use that context to inform their responses.
- These agents often use a ReAct-style logic but are built with prompts that are specifically designed to handle chat history.
- The prompt template includes a placeholder for conversation history, which is managed by a memory object.

Choosing the Right Agent

- **Choosing the Right Agent and Executor Configuration**

- The choice of agent type depends on your application's requirements:
- **ReAct:** Use for general-purpose, **complex tasks** that require **flexible tool usage** and a **clear reasoning path**. It's a **strong default choice**.
- **Self-Ask with Search:** Ideal for **fact-based, multi-hop question-answering** where **transparency of the reasoning process** is important.
- **Conversational:** Necessary for **building chatbots, assistants, or any application that needs to maintain context over multiple turns of a conversation**.

Overview of Opensource Small language Models



Source: [Lu et al., 2024](#)

Detailed configurations of SLMs benchmarked

Affiliation	Model name	Size	Date	Attention	Layer number	Hidden size	Head num	Activation	Vocab. size	Open training datasets	Max context window
Meta 	OPT [28]	125M	2022.05	MHA	12	768	12	ReLU	50k	✓	2k
		350M			24	1024	16				
		1.3B			24	2048	32				
		2.7B			32	2560	32				
	Galactica [29]	125M 1.3B	2022.11	MHA	12 24	768 2048	12 32	GELU	50k		2k
BigScience 	Llama-3.2 [59]	3B	2024.09	GQA	28	3072	24	SiLU	128k		131k
		1B	2024.09	GQA	16	2048	32	SiLU	128k		131k
	Bloom [15]	560M 1.1B	2022.11	MHA	24	1024 1536	16	GELU _{tanh}	251k	✓	2k
	Bloomz [16]	1.1B 560M	2022.11	MHA		1536 1024					
EleutherAI 	Pythia [27]	160M	2023.03	MHA	12	768	12	GELU	50k	✓	2k
		410M			24	1024	16				
		1B			16	2048	8				
		1.4B			24	2048	16				
		2.8B			32	2560	32				
Cerebras 	Cerebras-GPT [18]	111M	2023.03	MHA	10	768	12	GELU	50k	✓	2k
		256M			14	1088	17				
		590M			18	1536	12				
		1.3B			24	2048	16				
		2.7B			32	2560	32				
Microsoft 	Phi-1 [60]	1.3B	2023.09	MHA	24	2048	32	GELU _{tanh}	51k		2k
	Phi-1.5 [61]	1.3B	2023.09	MHA	24	2048	32	GELU _{tanh}	51k		
	Phi-2 [62]	2.7B	2023.12	MHA	32	2560	32	GELU _{tanh}	51k		2k
	Phi-3-mini* [63]	3.8B	2024.04	MHA	32	3072	32	SiLU	32k		4k
	Phi-3.5-mini*	2.7B	2024.09	MHA	32	3072	32	SiLU	32k		4k

Source- see complete list here: [Lu et al., 2024](#)

Summary

- The model will choose which results will be interesting to our query.
- The model can access different information but has nowhere to save it.
- Using an LLM for internet research can be useful for different fields (from medicine to finance).
 - For example, models have been used to access genetic sequences and conduct comparisons, or to search for information about drugs, chemical structures, and more.
 - Similarly, a model can search for the latest financial and economic news.
- In the RAG, we will discuss how the **model can save and access this memory**.
 - For example, we may want our model to be able to access conversation history or extend its knowledge.
 - This can be useful in both business applications, but also in fields such as finance and healthcare

Creating Web Search Agent using LangChain

DEMO